FXL

Đặc tả ngôn ngữ lập trình

Mục 1: FXL cơ bản (sơ khai)

Thuộc tính	Giá trị
Tên ngôn ngữ	FXL
Loại	General-purpose programming language
Định hướng	Application, UI, Graphics và General Programming
Compiler target giai đoạn đầu	C++

1. Giới thiệu về FXL

FXL là một ngôn ngữ lập trình đa mục đích (General-purpose programming language), được thiết kế nhằm giúp việc xây dựng ứng dụng, giao diện người dùng (UI), đồ họa (Graphics) và các chương trình thông thường trở nên đơn giản, trực quan và dễ viết hơn so với việc sử dụng C++ trực tiếp.

FXL không phải là ngôn ngữ chỉ dành cho UI.

Mục tiêu của FXL là trở thành một ngôn ngữ lập trình có thể sử dụng cho nhiều loại chương trình khác nhau, đồng thời cung cấp cú pháp thuận tiện cho những lĩnh vực như:

- Application development
- UI development
- 2D / 3D Graphics
- General programming
- Các hệ thống khác sẽ được phát triển trong tương lai

Ở giai đoạn đầu, FXL được thiết kế để biên dịch sang C++. Quá trình cơ bản:

```
FXL Source
    ↓
Compiler
    ↓
C++ Source
```

C++ được sử dụng làm compilation target ban đầu để FXL có thể tận dụng hệ sinh thái và khả năng của C++ trong khi vẫn xây dựng một cú pháp và hệ thống ngôn ngữ riêng.

2. Hình thức của FXL

Một chương trình FXL cơ bản có thể có dạng:

```FXL
import ui

window = Window()
window.setSize(800, 600)

button = Button()
button.setText("Click me")

window.add(button)
```

Cấu trúc cơ bản:

```
import <thư_viện>

<biến> = <tạo_object>

<biến>.<thuộc_tính_của_biến_dựa_trên_biến_đã_đặt>(<chi_tiết_theo_thuộc_tính>)
```

Trong đó:

- `import` dùng để đưa thư viện vào chương trình.
- `<biến>` là tên được dùng để tham chiếu đến object.
- `<tạo_object>` là thao tác tạo object.
- `<thuộc_tính_của_biến_dựa_trên_biến_đã_đặt>` biểu diễn method hoặc thao tác được thực hiện trên object.
- `<chi_tiết_theo_thuộc_tính>` là các tham số cần thiết cho thao tác đó.

Ví dụ:

```FXL
window.setSize(800, 600)
```

Thành phần	Ý nghĩa
`window`	Object đang được thao tác
`setSize`	Thao tác thay đổi kích thước
`800, 600`	Thông tin được truyền vào thao tác

3. Object và Object Block

FXL cho phép tạo object trực tiếp:

```FXL
window = Window()
```

Object cũng có thể được cấu hình ngay trong quá trình tạo bằng **Object Block**:

```FXL
window = Window(
    window.setSize(800, 600)
)
```

Một Object Block có thể chứa nhiều thao tác:

```FXL
window = Window(
    window.setSize(800, 600)
    window.setTitle("My Application")
)
```

Object có thể tiếp tục chứa object khác:

```FXL
window = Window(
    window.setSize(800, 600)

    rectangle = Object(
        rectangle.setColor(#000000)
        rectangle.setOpacity(15%)
        rectangle.setSize(13, 45)
    )
)
```

Cấu trúc trên có thể được hiểu là:

```
Window
└── rectangle
```

Object có thể tiếp tục được lồng nhiều cấp tùy theo cấu trúc chương trình. Ví dụ:

```
Window
└── Panel
    └── Rectangle
        └── Button
```

Object Block và cấu trúc lồng nhau là nền tảng để FXL biểu diễn quan hệ giữa các object.

4. Hình thức cơ bản của lệnh rút gọn FXL

Trong FXL, một số thao tác có thể được viết ngắn hơn thông qua hệ thống `#auto...`. Ví dụ thông thường:

```FXL
import ui

window = Window(
    window.setSize(800, 600)

    rectangle = Object(
        rectangle.setColor(#000000)
        rectangle.setOpacity(15%)
        rectangle.setSize(13, 45)
    )
)
```

Việc phải lặp lại `window.set...` và `rectangle.set...` có thể làm code dài và lặp lại. FXL vì vậy có hệ thống **Auto Directive** để cung cấp các cách viết rút gọn.

5. #autoSet

`#autoSet` là một Auto Directive dùng để rút gọn cách viết các thao tác thuộc nhóm `set`. Ví dụ:

```FXL
import ui

main #autoSet

window = Window(
    Size(800, 600)

    rectangle = Object(
        Color(#000000)
        Opacity(15%)
        Size(13, 45)
    )
)
```

Các câu lệnh rút gọn trên có ý nghĩa tương ứng với:

```FXL
window.setSize(800, 600)

rectangle.setColor(#000000)
rectangle.setOpacity(15%)
rectangle.setSize(13, 45)
```

Mục đích của `#autoSet` là loại bỏ phần cú pháp lặp lại liên quan đến setter và object hiện tại.

6. Các hình thức khai báo #autoSet

`#autoSet` có thể được sử dụng theo dạng **khai báo**:

```FXL
import ui

main #autoSet

window = Window(
    Size(800, 600)

    rectangle = Object(
        Color(#000000)
        Opacity(15%)
        Size(13, 45)
    )
)
```

Hoặc sử dụng dưới dạng **block**:

```FXL
import ui

#autoSet {
    window = Window(
        Size(800, 600)

        rectangle = Object(
            Color(#000000)
            Opacity(15%)
            Size(13, 45)
        )
    )
}
```

Block `#autoSet` xác định phạm vi mà cơ chế `#autoSet` được áp dụng.

7. Các Auto Directive khác

`#autoSet` chỉ là một loại Auto Directive. Ngoài các thao tác thuộc nhóm `set`, FXL còn có thể có những loại hành động khác cần cơ chế rút gọn riêng. Vì vậy trong tương lai FXL có thể có thêm những Auto Directive khác. Ví dụ:

```FXL
#autoPick {
    ...
}
```

`#autoPick` chỉ là ví dụ về một Auto Directive khác. Hành vi và mục đích cụ thể của nó sẽ được định nghĩa trong specification tương ứng khi tính năng này được thiết kế.

8. Nhiều Auto Directive trong cùng một file

Một file FXL **có thể chứa nhiều Auto Directive khác nhau**. Mỗi Auto Directive được khai báo bằng một block riêng và có phạm vi áp dụng riêng. Ví dụ:

```FXL
import ui

#autoSet {
    window = Window(
        Size(800, 600)
    )
}

#autoPick {
    ...
}
```

Cách viết trên là hợp lệ.

- `#autoSet` chỉ áp dụng cho block của nó.
- `#autoPick` chỉ áp dụng cho block của nó.

- Các Auto Directive không tự động ảnh hưởng đến những block khác.

9. Không được kết hợp nhiều Auto Directive vào cùng một khai báo

Các Auto Directive không được ghép trực tiếp với nhau trong cùng một khai báo. Ví dụ sau là **không hợp lệ**:

```FXL
import ui

#autoSet #autoPick {
    window = Window(
        Size(800, 600)
    )
}
```

Mỗi Auto Directive phải được khai báo riêng:

```FXL
#autoSet {
    ...
}

#autoPick {
    ...
}
```

Đây là hai block độc lập và mỗi block có phạm vi áp dụng riêng.

10. Lồng Auto Directive

Các Auto Directive của FXL có thể được lồng vào nhau. Khi một Auto Directive được đặt bên trong một Auto Directive khác, directive bên trong tạo ra một **vùng ảnh hưởng riêng** cho phần code nằm trong block của nó. Ví dụ:

```FXL
import ui

#autoSet {
    window = Window(
        Size(800, 600)

        #autoPick {
            rectangle = Object(
                Color(#000000)
                Opacity(15%)
                Size(13, 45)
            )
        }
    )
}
```

Phạm vi ảnh hưởng được xác định như sau:

Đoạn code	Vùng chịu ảnh hưởng của
`Size(800, 600)`	`#autoSet`
`Color(#000000)`	`#autoPick`
`Opacity(15%)`	`#autoPick`
`Size(13, 45)`	`#autoPick`

Do đó các thao tác nằm trong block `#autoPick` được xử lý theo quy tắc của `#autoPick`, không phải theo quy tắc riêng của `#autoSet`.

10.1. Phạm vi của Auto Directive lồng nhau

Mỗi block `#auto...` xác định một vùng ảnh hưởng riêng. Ví dụ:

```FXL
#autoSet {
    ...

    #autoPick {
        ...
    }

    ...
}
```

Có thể hiểu thành:

```
#autoSet
├── code chịu ảnh hưởng của #autoSet
│
├── #autoPick
│   └── code chịu ảnh hưởng của #autoPick
│
└── code tiếp tục chịu ảnh hưởng của #autoSet
```

Khi compiler đi vào block `#autoPick`, cơ chế xử lý của `#autoPick` được áp dụng cho vùng đó. Khi compiler đi ra khỏi block `#autoPick`, phạm vi xử lý quay trở lại Auto Directive của block bên ngoài.

10.2. Auto Directive bên trong tạo phạm vi mới

Auto Directive bên trong không làm mất cấu trúc scope của directive bên ngoài. Thay vào đó, nó tạo ra một **inner auto scope**. Ví dụ:

```FXL
#autoSet {
    A()

    #autoPick {
        B()
    }

    C()
}
```

Lệnh	Directive xử lý
`A()`	`#autoSet`
`B()`	`#autoPick`
`C()`	`#autoSet`

Vì vậy `#autoPick` chỉ tác động đến vùng được bao bởi block của nó.

10.3. Auto Directive có thể tiếp tục được lồng

Hệ thống Auto Directive được thiết kế để có thể mở rộng và cho phép các directive được lồng theo cấu trúc block khi specification của từng directive cho phép. Ví dụ về mặt cấu trúc:

```FXL
#autoSet {
    ...

    #autoPick {
        ...

        #autoSomething {
            ...
        }

        ...
    }

    ...
}
```

Mỗi block tạo ra một phạm vi xử lý tương ứng. Quy tắc cụ thể về việc các Auto Directive tương tác với nhau sẽ được định nghĩa trong specification của từng directive.

10.4. Nguyên tắc phạm vi

FXL sử dụng nguyên tắc:

> ***Auto Directive chỉ ảnh hưởng đến vùng được bao bởi block của chính nó.***

Nếu một Auto Directive được lồng bên trong một Auto Directive khác, directive bên trong tạo ra một vùng ảnh hưởng mới. Sau khi kết thúc block bên trong, phạm vi xử lý quay trở lại block bên ngoài.

11. Phạm vi của Auto Directive

Auto Directive chỉ áp dụng trong phạm vi mà nó được khai báo. Ví dụ:

```FXL
#autoSet {
    window = Window(
        Size(800, 600)
    )
}

window.setTitle("FXL")
```

`Size(800, 600)` nằm trong block `#autoSet`, vì vậy có thể sử dụng cơ chế rút gọn của `#autoSet`. Trong khi đó, `window.setTitle("FXL")` nằm ngoài block nên không chịu ảnh hưởng của `#autoSet`.

12. Quy tắc của #autoSet

`#autoSet` không có nhiệm vụ tự động biến mọi hành động thành setter. Chỉ những hành động đã được xác định thuộc hệ thống `#autoSet` mới được rút gọn.

Ví dụ, `Size(800, 600)` có thể được ánh xạ thành `setSize(800, 600)` và dựa trên object hiện tại thành `window.setSize(800, 600)`.

Tương tự, `Color(#000000)` có thể được ánh xạ thành `setColor(#000000)` và thành `rectangle.setColor(#000000)`.

13. Setter Mapping

Một số mapping ban đầu của `#autoSet`:

FXL	Setter tương ứng
`Size(...)`	`setSize(...)`
`Color(...)`	`setColor(...)`
`Opacity(...)`	`setOpacity(...)`
`Position(...)`	`setPosition(...)`
`Text(...)`	`setText(...)`

Danh sách này có thể được mở rộng khi FXL phát triển. Compiler phải dựa trên mapping được định nghĩa bởi FXL. Compiler không được tự ý coi một hành động bất kỳ là setter chỉ dựa vào tên.

14. Không được tự động bỏ những hành động khác

Khi chỉ khai báo `#autoSet`, chỉ những hành động thuộc phạm vi của `#autoSet` mới được rút gọn. Các hành động khác không được tự động bỏ hoặc tự động biến đổi. Ví dụ:

```
FXL
Rotate(30)
```

Nếu `Rotate` chưa được định nghĩa là một thao tác thuộc `#autoSet`, compiler không được tự động biến nó thành `setRotate(30)` hay tự suy luận một cách xử lý khác. Nói cách khác:

> ***#autoSet chỉ xử lý những gì thuộc #autoSet.***

Những Auto Directive khác sẽ chịu trách nhiệm cho những loại hành động khác nếu chúng được thiết kế trong tương lai.

15. Object Scope và #autoSet

`#autoSet` phải xác định object đang được thao tác dựa trên cấu trúc và scope của chương trình. Ví dụ:

```FXL
#autoSet {
    window = Window(
        Size(800, 600)

        rectangle = Object(
            Color(#000000)
            Size(13, 45)
        )
    )
}
```

Lệnh	Object hiện tại	Kết quả tương ứng
`Size(800, 600)`	`window`	`window.setSize(800, 600)`
`Color(#000000)`	`rectangle`	`rectangle.setColor(#000000)`
`Size(13, 45)`	`rectangle`	`rectangle.setSize(13, 45)`

Vì vậy `#autoSet` không chỉ là việc thay thế chuỗi ký tự. Compiler cần hiểu cấu trúc object và scope của chương trình.

16. Khái niệm Compiler của FXL

Compiler của FXL là chương trình chịu trách nhiệm chuyển source code FXL thành source code C++ ở giai đoạn đầu. Mục tiêu của compiler là:

```
FXL Source
    ↓
Phân tích
    ↓
Hiểu cấu trúc chương trình
    ↓
Kiểm tra ý nghĩa
    ↓
Xử lý các tính năng của FXL
    ↓
Sinh C++ Source
```

Compiler phải hiểu cú pháp và quy tắc của FXL thay vì chỉ thực hiện việc thay thế văn bản đơn giản.

17. Compiler Pipeline

Pipeline cơ bản của FXL ở giai đoạn đầu:

```
FXL Source
    ↓
Lexer
    ↓
Tokens
    ↓
Parser
    ↓
AST
    ↓
Semantic Analysis
    ↓
Auto / Language Resolution
    ↓
C++ Code Generator
    ↓
C++ Source
```

18. Lexer

Lexer là thành phần đầu tiên xử lý source code FXL. Lexer đọc source code và chia nó thành các **Token**. Ví dụ, `Size(800, 600)` có thể được phân tích thành:

Loại token	Giá trị
Identifier	`Size`
LeftParenthesis	`(`
Number	`800`
Comma	`,`
Number	`600`
RightParenthesis	`)`

Lexer chỉ tập trung vào việc nhận diện các thành phần cơ bản của source code. Lexer không chịu trách nhiệm quyết định đầy đủ ý nghĩa của câu lệnh.

19. Parser

Parser nhận danh sách token từ Lexer và phân tích cấu trúc cú pháp của chương trình. Ví dụ:

```FXL
window = Window(
    Size(800, 600)
)
```

Parser phải nhận biết được rằng:

Thành phần	Vai trò
`window`	Biến
`Window()`	Object creation
`Size(800, 600)`	Một thao tác nằm trong Object Block

Parser sau đó xây dựng cấu trúc AST tương ứng.

20. AST

AST là viết tắt của **Abstract Syntax Tree**. AST là cấu trúc dữ liệu biểu diễn chương trình sau khi Parser phân tích cú pháp. Ví dụ về mặt khái niệm:

```
Program
└── ObjectDeclaration
    ├── Name: window
    ├── Type: Window
    └── Body
        └── Call
            ├── Name: Size
            └── Arguments
                ├── 800
                └── 600
```

AST giúp các bước phía sau compiler làm việc với cấu trúc chương trình thay vì phải xử lý source code dưới dạng chuỗi.

21. Semantic Analysis

Sau khi Parser tạo AST, compiler cần kiểm tra **ý nghĩa** của chương trình. Giai đoạn này được gọi là Semantic Analysis. Semantic Analysis có thể kiểm tra những vấn đề như:

- Biến có hợp lệ hay không.
- Object có được tạo đúng hay không.
- Cấu trúc scope có hợp lệ hay không.
- Một thao tác có được phép sử dụng trên object hay không.
- `#autoSet` có được áp dụng đúng phạm vi hay không.
- Setter mapping có tồn tại hay không.

Ví dụ:

```FXL
#autoSet {
    window = Window(
        Size(800, 600)
    )
}
```

Semantic Analysis cần xác định:

```
Current Object = window
Size
  ↓
Setter mapping tồn tại
  ↓
setSize
  ↓
window.setSize(800, 600)
```

22. Auto Resolution

Sau khi compiler đã có AST và thông tin semantic cần thiết, những Auto Directive như `#autoSet` có thể được xử lý. Với ví dụ:

```FXL
#autoSet {
    window = Window(
        Size(800, 600)
    )
}
```

Trước khi xử lý `#autoSet`, AST có thể biểu diễn:

```
ObjectDeclaration
└── window
    └── AutoSetAction
        ├── Name: Size
        └── Arguments:
            ├── 800
            └── 600
```

Sau khi resolve:

```
ObjectDeclaration
└── window
    └── MethodCall
        ├── Target: window
        ├── Method: setSize
        └── Arguments:
            ├── 800
            └── 600
```

Như vậy compiler đã chuyển cú pháp rút gọn của FXL thành cấu trúc mà C++ Generator có thể xử lý.

23. C++ Code Generator

C++ Code Generator là thành phần cuối của pipeline FXL trong giai đoạn đầu. Nó nhận AST đã được xử lý và sinh ra C++ Source. Ví dụ FXL:

```FXL
window = Window(
    Size(800, 600)
)
```

Sau quá trình xử lý có thể tương ứng với C++:

```C++
auto window = Window();
window.setSize(800, 600);
```

C++ được sinh ra sau đó có thể tiếp tục được biên dịch bằng C++ compiler để tạo chương trình cuối cùng. Pipeline đầy đủ:

```
FXL
 ↓
Lexer
 ↓
Parser
 ↓
AST
 ↓
Semantic Analysis
 ↓
Auto Resolution
 ↓
C++ Generator
 ↓
C++
 ↓
C++ Compiler
 ↓
Executable
```

24. Nguyên tắc của Compiler FXL

Compiler FXL phải ưu tiên việc xử lý đúng specification của ngôn ngữ. Compiler không được tự ý tạo ra những hành vi chưa được định nghĩa chính thức. Đặc biệt đối với những tính năng như `#autoSet`:

Trạng thái	Compiler xử lý
Đã được định nghĩa	Compiler xử lý
Chưa được định nghĩa	Compiler không tự suy đoán

Điều này giúp FXL có hành vi nhất quán khi ngôn ngữ phát triển.

25. Ví dụ hoàn chỉnh của Mục 1

Một chương trình FXL sử dụng các khái niệm cơ bản:

```FXL
import ui

#autoSet {
    window = Window(
        Size(800, 600)

        rectangle = Object(
            Color(#000000)
            Opacity(15%)
            Size(13, 45)
        )
    )
}
```

Về mặt cấu trúc:

```
Program
│
├── import ui
│
└── #autoSet
    │
    └── window = Window
        │
        ├── Size(800, 600)
        │
        └── rectangle = Object
            │
            ├── Color(#000000)
            ├── Opacity(15%)
            └── Size(13, 45)
```

Trong quá trình compiler xử lý:

```
FXL Source
    ↓
Lexer
    ↓
Tokens
    ↓
Parser
    ↓
AST
    ↓
Semantic Analysis
    ↓
#autoSet Resolution
    ↓
C++ Code Generator
    ↓
C++ Source
```

Đây là nền tảng ban đầu để FXL tiếp tục phát triển thành một ngôn ngữ lập trình hoàn chỉnh.

26. Trạng thái của Mục 1

Mục 1 hiện mới xác định nền tảng sơ khai của FXL. Đã xác định:

- FXL là general-purpose programming language.
- FXL hướng tới Application, UI, Graphics và General Programming.
- Cú pháp import cơ bản.
- Cú pháp tạo object.
- Object Block.
- Nested Object.
- Auto Directive.
- `#autoSet`.
- Phạm vi của Auto Directive.
- Setter mapping cơ bản.
- Compiler Pipeline: Lexer, Parser, AST, Semantic Analysis, Auto Resolution, C++ Code Generation.

Các hệ thống chưa được định nghĩa trong Mục 1 sẽ được thiết kế ở những mục specification khác khi FXL phát triển.

FXL

Đặc tả ngôn ngữ lập trình

Mục 2: FXL UI Runtime & Rendering

Thuộc tính	Giá trị
Tên mục	FXL UI Runtime & Rendering
Mục tiêu	Xây dựng runtime UI và hệ thống rendering thực nghiệm để FXL có thể tạo và hiển thị giao diện thật
Trọng tâm	UI Runtime, Object Tree, Window, Basic Rendering và Preview
Compiler target	C++

Chạy thực nghiệm và dựng giao diện cơ bản của FXL.

1. Giới thiệu về FXL UI Runtime & Rendering

Trong Mục 1, FXL đã xác định cú pháp cơ bản và compiler pipeline:

```
FXL Source
    ↓
Lexer
    ↓
Parser
    ↓
AST
    ↓
Semantic Analysis
    ↓
Auto Resolution
    ↓
C++ Code Generator
    ↓
C++
    ↓
C++ Compiler
    ↓
Executable
```

Mục 2 mở rộng hệ thống FXL bằng cách xây dựng **FXL UI Runtime** và **Rendering System**. Mục tiêu của Mục 2 là đưa FXL từ trạng thái:

> *Có thể biên dịch các khai báo UI thành chương trình C++*

sang trạng thái:

> *Có thể thực sự tạo và hiển thị một giao diện UI trên màn hình.*

Pipeline được mở rộng:

```
FXL Source
    ↓
FXL Compiler
    ↓
Generated C++
    ↓
FXL UI Runtime
    ↓
UI Object
    ↓
Rendering
    ↓
Native Window / Display
```

Runtime là phần chịu trách nhiệm thực thi các object UI và kết nối chúng với hệ thống hiển thị.

- Compiler chịu trách nhiệm hiểu FXL.
- Runtime chịu trách nhiệm thực thi các object mà chương trình FXL tạo ra.
- Rendering chịu trách nhiệm chuyển trạng thái UI thành hình ảnh hiển thị.

2. Phân biệt Compiler, Runtime và Rendering

FXL phân chia hệ thống thành các thành phần khác nhau.

2.1. Compiler

Compiler chịu trách nhiệm:

- Đọc source FXL.
- Phân tích cú pháp.
- Phân tích semantic.
- Xử lý Auto Directive.
- Sinh C++.

Compiler không trực tiếp chịu trách nhiệm vẽ từng pixel của giao diện.

2.2. Runtime

Runtime chịu trách nhiệm:

- Tạo object UI.
- Quản lý object.
- Quản lý quan hệ giữa các object.
- Áp dụng thuộc tính của object.
- Kết nối object với hệ thống UI và rendering.

2.3. Rendering

Rendering chịu trách nhiệm biến trạng thái của UI thành hình ảnh được hiển thị. Ví dụ:

```
Rectangle
    ↓
Position
    ↓
Size
    ↓
Color
    ↓
Renderer
    ↓
Pixels
```

Ba thành phần này tạo thành một pipeline hoàn chỉnh nhưng có trách nhiệm khác nhau.

3. FXL UI Runtime

FXL UI Runtime là runtime dùng để thực thi các thành phần UI được tạo từ chương trình FXL. Runtime là tầng nằm giữa generated C++ và hệ thống hiển thị. Cấu trúc khái niệm:

```
FXL
    ↓
Compiler
    ↓
C++
    ↓
FXL UI Runtime
    ↓
UI Object System
    ↓
Rendering System
    ↓
Operating System / Graphics System
```

Runtime không thay thế compiler và không phân tích cú pháp FXL. Runtime nhận kết quả đã được compiler chuyển thành C++ và thực thi các thao tác tương ứng.

4. UI Object

UI Object là một object có khả năng tham gia vào hệ thống giao diện của FXL. Ví dụ:

```FXL
window = Window()

button = Button()

rectangle = Rectangle()

panel = Panel()
```

Các object này có thể được tạo và sử dụng trong chương trình FXL. Ở giai đoạn đầu, FXL xác định một số UI object cơ bản:

Object	Vai trò ban đầu
`Window`	Cửa sổ chứa giao diện
`Panel`	Container cho các object UI khác
`Rectangle`	Primitive hình chữ nhật
`Button`	Control tương tác cơ bản
`Object`	Object cơ bản dùng làm nền tảng mở rộng

Danh sách trên là nền tảng ban đầu của UI Runtime và có thể được mở rộng khi FXL phát triển.

5. Object Tree

Các UI Object có thể tạo thành một cấu trúc phân cấp. Ví dụ:

```FXL
window = Window(
    panel = Panel(
        rectangle = Rectangle(
            button = Button()
        )
    )
)
```

Cấu trúc có thể được biểu diễn:

```
Window
└── Panel
    └── Rectangle
        └── Button
```

Cấu trúc này được gọi là **Object Tree**, biểu diễn quan hệ giữa các object trong giao diện. Mỗi object có thể có:

- Object cha.
- Các object con.
- Các thuộc tính của chính object đó.

6. Parent và Child

Trong Object Tree:

- Object chứa một object khác được xem là **parent** trong cấu trúc UI.
- Object nằm bên trong được xem là **child**.

Ví dụ:

```FXL
window = Window(
    button = Button()
)
```

```
Window
└── Button
```

Trong trường hợp này `Window` là parent và `Button` là child. Runtime phải duy trì được cấu trúc object tree để rendering có thể xử lý giao diện theo đúng cấu trúc chương trình.

7. Window

`Window` là UI Object đại diện cho cửa sổ hiển thị chính của chương trình. Ví dụ:

```FXL
window = Window()
```

Window có thể có các thuộc tính ban đầu như:

```FXL
window.setSize(800, 600)
```

Hoặc sử dụng `#autoSet`:

```FXL
#autoSet {
    window = Window(
        Size(800, 600)
    )
}
```

Window phải được runtime tạo thành một cửa sổ thật khi chương trình được chạy ở chế độ UI. Mục tiêu thực nghiệm ban đầu:

```
window = Window()
    ↓
FXL Runtime
    ↓
Native Window
    ↓
Hiển thị trên màn hình
```

8. Window Size

`Size(...)` được sử dụng để xác định kích thước của UI Object khi thao tác này được áp dụng cho object có hỗ trợ kích thước. Ví dụ:

```FXL
window.setSize(800, 600)
```

Trong `#autoSet`:

```FXL
#autoSet {
    window = Window(
        Size(800, 600)
    )
}
```

Kết quả semantic tương ứng:

```
window.setSize(800, 600)
```

Runtime phải phản ánh kích thước này vào object thật. Đối với `Window`, kích thước phải ảnh hưởng đến kích thước cửa sổ hiển thị.

9. Position

`Position(...)` dùng để xác định vị trí của một UI Object khi object hỗ trợ thuộc tính này. Ví dụ:

```FXL
rectangle.setPosition(20, 30)
```

Hoặc:

```FXL
#autoSet {
    rectangle = Rectangle(
        Position(20, 30)
    )
}
```

Runtime phải lưu giữ thông tin vị trí của object để renderer sử dụng. Ở giai đoạn đầu, hệ tọa độ 2D được sử dụng cho UI cơ bản.

10. Size của UI Object

Ngoài Window, các UI Object khác cũng có thể có kích thước. Ví dụ:

```FXL
button.setSize(200, 60)

rectangle.setSize(400, 250)
```

Kích thước của object được lưu trong runtime và được renderer sử dụng khi xác định vùng hiển thị.

11. Rectangle

`Rectangle` là primitive rendering đầu tiên của FXL. Mục tiêu của Rectangle là cung cấp một thành phần đơn giản để kiểm tra toàn bộ pipeline:

```
FXL
    ↓
Compiler
    ↓
Runtime
    ↓
Renderer
    ↓
Visible Result
```

Ví dụ:

```FXL
import ui

window = Window(
    window.setSize(800, 600)

    rectangle = Rectangle(
        rectangle.setSize(400, 250)
        rectangle.setColor(#202020)
    )
)
```

Sau khi chạy, renderer phải hiển thị một hình chữ nhật thật trên cửa sổ. Rectangle là một trong những object thực nghiệm quan trọng nhất của Mục 2.

12. Color

UI Object có thể có thuộc tính màu. Ví dụ:

```FXL
rectangle.setColor(#202020)
```

Hoặc:

```FXL
#autoSet {
    rectangle = Rectangle(
        Color(#202020)
    )
}
```

Màu được lưu trong runtime object và được renderer sử dụng khi vẽ. Màu không chỉ được ghi nhận dưới dạng console output mà phải ảnh hưởng đến hình ảnh hiển thị thật.

13. Opacity

UI Object có thể có thuộc tính độ trong suốt. Ví dụ:

```FXL
rectangle.setOpacity(50%)
```

Hoặc:

```FXL
#autoSet {
    rectangle = Rectangle(
        Opacity(50%)
    )
}
```

Runtime phải lưu giá trị opacity và renderer phải sử dụng giá trị đó trong quá trình hiển thị. Ví dụ:

Giá trị	Ý nghĩa
`Opacity(100%)`	Object hoàn toàn không trong suốt
`Opacity(50%)`	Object có mức trong suốt tương ứng

14. Button

`Button` là control UI cơ bản đầu tiên của FXL. Ví dụ:

```FXL
button = Button()

button.setText("Hello from FXL")

button.setSize(200, 60)
```

Button phải được runtime tạo thành một UI object thực tế. Ở giai đoạn đầu, Button cần hỗ trợ tối thiểu:

- Kích thước.
- Vị trí.
- Nội dung text.
- Hiển thị trên Window.

Ví dụ:

```FXL
import ui

window = Window(
    window.setSize(800, 600)

    button = Button(
        button.setText("Hello from FXL")
        button.setSize(200, 60)
    )
)
```

Khi preview chương trình, Button phải được hiển thị trong cửa sổ thật.

15. Text

Text là dữ liệu hiển thị của một số UI Object. Ví dụ:

```FXL
button.setText("Hello from FXL")
```

Trong `#autoSet`:

```FXL
#autoSet {
    button = Button(
        Text("Hello from FXL")
    )
}
```

Runtime lưu trữ nội dung text. Renderer chịu trách nhiệm hiển thị text lên màn hình. Hệ thống typography nâng cao chưa được định nghĩa trong Mục 2 và sẽ được thiết kế trong specification tương ứng khi cần thiết.

16. Rendering System

Rendering System là thành phần biến trạng thái của UI Object thành hình ảnh. Ví dụ:

```
Rectangle
    ↓
Position
    ↓
Size
    ↓
Color
    ↓
Renderer
    ↓
Displayed Rectangle
```

Rendering System không thay đổi logic của compiler. Renderer nhận dữ liệu từ UI Runtime.

17. Renderer và UI Object

UI Object không nên trực tiếp chứa toàn bộ implementation của renderer. Thay vào đó, cấu trúc khái niệm:

```
UI Object
    ↓
UI Runtime
    ↓
Renderer
    ↓
Graphics Backend
```

Điều này giúp UI Object và rendering system có thể được phát triển độc lập hơn. Ví dụ, `Button` không cần trực tiếp biết toàn bộ chi tiết của graphics API. Button chỉ cần cung cấp trạng thái cần thiết cho renderer.

18. Graphics Backend

Graphics Backend là tầng kết nối Renderer với hệ thống đồ họa của nền tảng. Cấu trúc khái niệm:

```
UI Object
    ↓
UI Runtime
    ↓
Renderer
    ↓
Graphics Backend
    ↓
Graphics API / Native Display System
```

Mục 2 xác định kiến trúc theo hướng backend có thể thay đổi. FXL không được buộc compiler phải hiểu trực tiếp một graphics API cụ thể.

19. OpenGL và GLSL

OpenGL và GLSL có thể được sử dụng trong quá trình thực nghiệm của FXL Rendering System khi được lựa chọn làm graphics backend phù hợp. Nếu sử dụng OpenGL:

```
FXL UI Runtime
    ↓
Renderer
    ↓
OpenGL
    ↓
GLSL Shader
    ↓
GPU
```

Mục đích ban đầu có thể là:

- Render Rectangle.
- Render màu.
- Render opacity.
- Render các primitive UI cơ bản.

OpenGL và GLSL thuộc về Rendering / Graphics Backend, không thuộc Lexer, Parser hoặc Semantic Analysis của FXL. FXL specification không coi OpenGL hoặc GLSL là một phần bắt buộc của cú pháp ngôn ngữ ở Mục 2.

20. Runtime State

Runtime phải lưu trữ trạng thái cần thiết của UI Object. Ví dụ:

Thuộc tính	Ví dụ
`Position`	`20, 30`
`Size`	`400, 250`
`Color`	`#202020`
`Opacity`	`50%`
`Text`	`"Hello from FXL"`

Không phải tất cả object đều bắt buộc phải có tất cả thuộc tính. Object chỉ sử dụng những thuộc tính mà nó hỗ trợ.

21. Runtime và Method

Các method mà compiler đã resolve có thể gọi vào UI Runtime. Ví dụ:

```FXL
window.setSize(800, 600)
```

Có thể được compiler sinh thành C++:

```C++
window.setSize(800, 600);
```

Runtime sau đó thực hiện thay đổi state của `window`. Ví dụ:

```FXL
rectangle.setColor(#202020)
```

Có thể được compiler sinh thành:

```C++
rectangle.setColor(...);
```

Runtime cập nhật màu của rectangle. Sau đó Renderer sử dụng state mới để render. Pipeline:

```
Method Call
    ↓
Runtime State Update
    ↓
Renderer
    ↓
Visible Result
```

21A. Vòng đời của FXL Runtime

FXL Runtime có một vòng đời xác định từ thời điểm chương trình bắt đầu thực thi cho đến khi chương trình kết thúc. Vòng đời Runtime được chia thành các giai đoạn:

```
Program Start
    ↓
Runtime Initialization
    ↓
Object Creation
    ↓
UI Tree Construction
    ↓
Window Initialization
    ↓
Rendering / Event Loop
    ↓
Program Shutdown
    ↓
Runtime Cleanup
    ↓
Program Exit
```

Mỗi giai đoạn có trách nhiệm riêng.

21A.1. Program Start

Khi executable được khởi chạy, chương trình bắt đầu từ entry point do C++ Code Generator tạo ra. Ví dụ về mặt khái niệm:

```C++
int main() {
    // FXL program
    return 0;
}
```

FXL Runtime được khởi tạo trong quá trình chương trình bắt đầu thực thi. Compiler không tạo một Runtime mới cho từng method call. Một chương trình FXL thông thường có một Runtime context chính trong suốt quá trình thực thi của chương trình.

21A.2. Runtime Initialization

Runtime Initialization là giai đoạn khởi tạo những thành phần cần thiết để chương trình UI có thể hoạt động. Tùy backend, Runtime có thể khởi tạo:

```
Runtime Context
    ↓
UI Object System
    ↓
Graphics / Window Backend
    ↓
Renderer
```

Ở giai đoạn này Runtime chuẩn bị môi trường để:

- Tạo UI Object.
- Tạo Window.

- Quản lý Object Tree.
- Khởi động hệ thống rendering.

Runtime Initialization phải xảy ra trước khi chương trình yêu cầu hiển thị UI.

21A.3. Object Creation

Sau khi Runtime được khởi tạo, generated C++ bắt đầu thực hiện các thao tác tạo object từ chương trình FXL. Ví dụ:

```FXL
window = Window()
button = Button()
rectangle = Rectangle()
```

Về mặt Runtime:

```
Window()     → tạo Window Object
Button()     → tạo Button Object
Rectangle()  → tạo Rectangle Object
```

Tại thời điểm này object được tạo nhưng chưa nhất thiết đã được hiển thị. Việc tạo object và việc render object là hai quá trình khác nhau.

21A.4. Object Initialization

Sau khi object được tạo, các thuộc tính và method đã được compiler resolve từ FXL được thực thi. Ví dụ:

```FXL
window.setSize(800, 600)
button.setText("Hello")
rectangle.setColor(#202020)
```

Runtime lần lượt cập nhật state của object:

```
Window
    Size = 800 × 600

Button
    Text = "Hello"

Rectangle
    Color = #202020
```

Các thay đổi này được lưu trong Runtime Object State. Object chỉ được renderer sử dụng sau khi state cần thiết đã được cập nhật.

21A.5. UI Tree Construction

Khi các object được thêm vào nhau, Runtime xây dựng Object Tree. Ví dụ:

```FXL
window = Window(
    panel = Panel(
        button = Button()
    )
)
```

Runtime có cấu trúc:

```
Window
└── Panel
    └── Button
```

Object Tree phải hoàn thành đủ thông tin cần thiết trước khi renderer xử lý UI. Mỗi object phải biết được quan hệ với object cha và object con theo cấu trúc Runtime đã định nghĩa.

21A.6. Window Initialization

Sau khi Window Object và các thành phần cần thiết đã được tạo, Runtime khởi tạo Window backend tương ứng. Ví dụ:

```
Window Object
    ↓
Window Backend
    ↓
Native Window
```

Kết quả là một cửa sổ thật được tạo trên hệ điều hành. Kích thước và các thuộc tính đã được thiết lập trước đó phải được phản ánh vào Window khi backend được khởi tạo.

21A.7. Renderer Initialization

Renderer được khởi tạo để chuẩn bị cho quá trình hiển thị UI. Tùy Graphics Backend, Renderer có thể tạo:

```
Rendering Context
Shader
Buffer
Texture Resource
Pipeline
```

Mục tiêu của giai đoạn này là tạo môi trường cần thiết để renderer có thể xử lý Object Tree. Renderer không tự tạo UI Object. UI Object do Runtime quản lý.

21A.8. Entering the Runtime Loop

Sau khi Runtime, Window và Renderer đã được khởi tạo, chương trình bước vào vòng lặp thực thi chính, gọi là **Runtime Loop**. Cấu trúc khái niệm:

```
Runtime Loop
    ↓
Process Events
    ↓
Update Runtime State
    ↓
Render UI
    ↓
Present Result
    ↓
Repeat
```

Vòng lặp tiếp tục cho đến khi chương trình nhận được điều kiện kết thúc.

21A.9. Event Processing

Trong mỗi vòng lặp, Runtime kiểm tra các event từ hệ điều hành hoặc UI backend. Ví dụ:

```
Window Close
Mouse Move
Mouse Press
Mouse Release
Keyboard Input
Resize
```

Event được tiếp nhận bởi Runtime. Ở giai đoạn đầu, Runtime chỉ cần có nền tảng để tiếp nhận và xử lý event. Cú pháp event chính thức của FXL chưa được định nghĩa trong Mục 2 thì không được tự ý thêm vào language syntax.

21A.10. Runtime State Update

Sau khi event được xử lý hoặc khi chương trình thực hiện thay đổi trạng thái, Runtime cập nhật state của các object. Ví dụ:

```
Mouse Input
    ↓
Button State
    ↓
Button Runtime State
    ↓
Renderer
```

Runtime State là nguồn dữ liệu mà Renderer sử dụng để tạo hình ảnh hiện tại của UI.

21A.11. Rendering

Sau khi Runtime State ở trạng thái cần thiết, Renderer thực hiện quá trình render. Pipeline cơ bản:

```
Object Tree
    ↓
Visible Objects
    ↓
Geometry
    ↓
Style / Properties
    ↓
Graphics Backend
    ↓
Rendered Image
```

Ví dụ:

```
Rectangle
    Position = (20, 20)
    Size = (400, 250)
    Color = #202020
    Opacity = 100%
```

Renderer sử dụng các giá trị trên để tạo rectangle tương ứng trên màn hình.

21A.12. Present

Sau khi frame được render, kết quả được đưa lên Window để người dùng nhìn thấy. Khái niệm:

```
Renderer
    ↓
Frame
    ↓
Present
    ↓
Window Display
```

Một frame hoàn chỉnh được coi là đã xử lý khi kết quả rendering đã được present tới cửa sổ hiển thị.

21A.13. Runtime Loop tiếp tục

Sau khi Present hoàn tất, Runtime quay lại Process Events và tiếp tục vòng lặp. Do đó, trong trạng thái chương trình đang chạy:

```
Process Events
    ↓
Update State
    ↓
Render
    ↓
Present
    ↓
Process Events
    ↓
...
```

Vòng lặp không kết thúc sau một lần render. Đây là điều phân biệt một UI Runtime đang hoạt động với một chương trình chỉ tạo ra một hình ảnh duy nhất.

21A.14. Program Termination

Runtime Loop kết thúc khi có điều kiện kết thúc chương trình. Ví dụ cơ bản:

```
Window Close
    ↓
Runtime nhận yêu cầu kết thúc
    ↓
Exit Runtime Loop
```

Tại thời điểm này Runtime không tiếp tục render frame mới.

21A.15. Runtime Shutdown

Sau khi Runtime Loop kết thúc, Runtime chuyển sang giai đoạn shutdown. Thứ tự khái niệm:

```
Stop Runtime Loop
    ↓
Stop Rendering
    ↓
Destroy Window
    ↓
Release UI Objects
    ↓
Release Graphics Resources
    ↓
Shutdown Runtime
```

Các tài nguyên được Runtime sở hữu phải được giải phóng theo đúng vòng đời của chúng.

21A.16. Resource Cleanup

Các tài nguyên Runtime có thể bao gồm:

```
Window
Graphics Context
Buffers
Textures
Shaders
UI Objects
Runtime State
```

Resource Cleanup phải đảm bảo:

- Không giữ tài nguyên sau khi chương trình kết thúc.
- Không truy cập object sau khi object đã bị hủy.
- Không tiếp tục render sau khi Renderer đã shutdown.
- Không sử dụng Graphics Backend sau khi backend đã được giải phóng.

21A.17. Program Exit

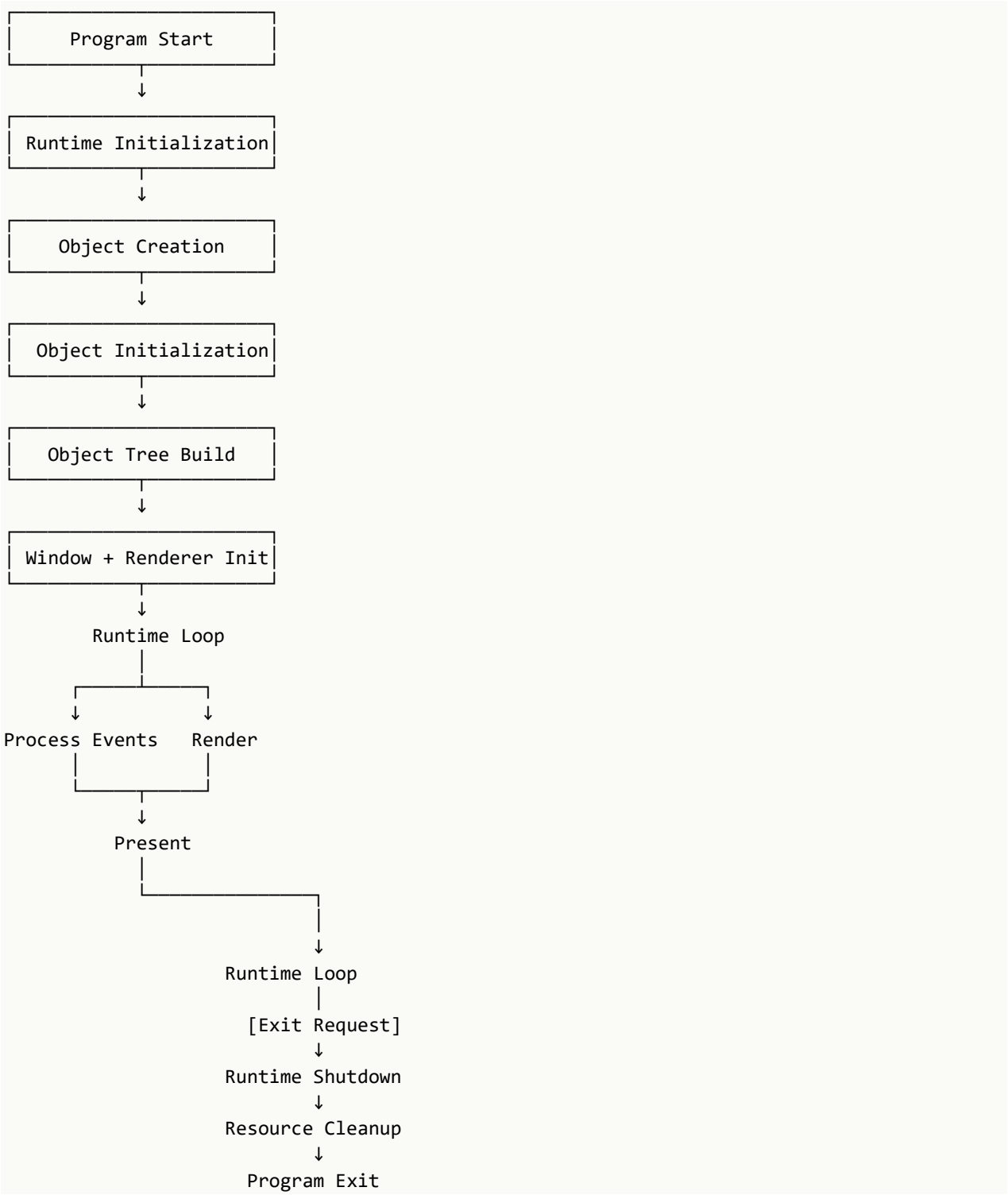
Sau khi Runtime Cleanup hoàn tất, chương trình trả quyền điều khiển về C++ entry point. Ví dụ:

```C++
int main() {
    // FXL Runtime
    return 0;
}
```

Khi `main()` kết thúc, executable kết thúc.

21A.18. Vòng đời tổng quát

Toàn bộ vòng đời Runtime có thể được biểu diễn:



21A.19. Quy tắc vòng đời

FXL Runtime tuân theo các nguyên tắc:

Giai đoạn	Trách nhiệm
Program Start	Bắt đầu chương trình
Runtime Initialization	Khởi tạo môi trường Runtime
Object Creation	Tạo UI Object
Object Initialization	Thiết lập state ban đầu
Object Tree Build	Xây dựng quan hệ parent/child
Window Initialization	Tạo native Window
Renderer Initialization	Chuẩn bị hệ thống rendering
Runtime Loop	Duy trì chương trình UI
Event Processing	Nhận và xử lý event
State Update	Cập nhật Runtime State
Rendering	Tạo frame
Present	Hiển thị frame
Shutdown	Dừng Runtime
Cleanup	Giải phóng tài nguyên
Program Exit	Kết thúc executable

21A.20. Quan hệ giữa Compiler và Runtime

Compiler và Runtime có vòng đời khác nhau. Compiler hoạt động trước khi executable được chạy:

```
FXL Source
    ↓
Compiler
    ↓
C++
    ↓
Executable
```

Runtime chỉ bắt đầu khi executable được chạy:

```
Executable Start
    ↓
Runtime Initialization
    ↓
UI Execution
    ↓
Runtime Loop
    ↓
Shutdown
```

Do đó:

> ***Compiler không phải Runtime.***

- Compiler tạo ra chương trình.
- Runtime thực thi chương trình.
- Renderer hiển thị trạng thái của chương trình.

21A.21. Quan hệ giữa Runtime và Preview

Preview sử dụng cùng vòng đời Runtime. Pipeline:

```
FXL Source
    ↓
Compiler
    ↓
Temporary Executable
    ↓
Runtime Initialization
    ↓
Object Creation
    ↓
Window Initialization
    ↓
Renderer
    ↓
Runtime Loop
    ↓
Visible UI
    ↓
Shutdown
```

Preview không bỏ qua Runtime Lifecycle. Preview phải khởi chạy Runtime thật để đảm bảo kết quả quan sát được tương ứng với behavior thực tế của ứng dụng FXL.

21A.22. Trạng thái Runtime

Runtime có thể được xem là đi qua các trạng thái:

```
Created
    ↓
Initializing
    ↓
Ready
    ↓
Running
    ↓
Stopping
    ↓
Shutdown
    ↓
Destroyed
```

Ý nghĩa:

Trạng thái	Ý nghĩa
Created	Runtime context đã được tạo nhưng chưa hoàn tất initialization.
Initializing	Runtime đang chuẩn bị UI backend, Window, Renderer và các thành phần cần thiết.
Ready	Runtime đã sẵn sàng để bắt đầu vòng lặp.
Running	Runtime đang thực hiện Event → Update → Render → Present.
Stopping	Runtime đã nhận yêu cầu kết thúc và đang thoát khỏi Runtime Loop.
Shutdown	Runtime đang giải phóng tài nguyên.
Destroyed	Runtime không còn hoạt động.

Không được sử dụng Runtime hoặc tài nguyên của Runtime sau trạng thái `Destroyed`.

21A.23. Mục tiêu của vòng đời Runtime

Việc định nghĩa vòng đời Runtime giúp FXL có một mô hình thống nhất cho:

- UI Object.
- Window.
- Renderer.
- Graphics Backend.
- Event System.
- Resource Management.
- Preview.
- Debugging.
- Shutdown.

Các hệ thống nâng cao sau này có thể mở rộng từ vòng đời này nhưng không được phá vỡ nguyên tắc:

```
Initialize
    ↓
Create
    ↓
Run
    ↓
Render / Update
    ↓
Stop
    ↓
Cleanup
    ↓
Exit
```

22. Preview

Preview là cơ chế cho phép chạy chương trình FXL và quan sát UI mà không cần đóng gói ứng dụng hoàn chỉnh. Ví dụ:

```bash
fxlc preview examples/preview_test.fxl
```

Preview phải sử dụng compiler thật của FXL. Pipeline:

```
FXL
    ↓
Compiler
    ↓
Generated C++
    ↓
C++ Compiler
    ↓
Executable
    ↓
FXL Runtime
    ↓
Renderer
    ↓
UI
```

Preview không được sử dụng một renderer giả chỉ để mô phỏng kết quả.

23. Preview và Runtime thật

Preview phải sử dụng cùng UI Runtime và Rendering System với chương trình FXL thực tế. Điều này giúp đảm bảo Preview và Application sử dụng cùng logic UI cơ bản. Preview chỉ là một cách chạy chương trình thuận tiện trong quá trình phát triển.

24. Chạy thực nghiệm

Mục 2 được phát triển theo phương pháp **thực nghiệm từng bước**. Không xây toàn bộ Rendering System trong một lần. Mỗi giai đoạn phải có một chương trình FXL thực tế để kiểm tra. Ví dụ:

Thực nghiệm	Nội dung	Kết quả mong đợi
Thực nghiệm 1	Window	Native Window
Thực nghiệm 2	Window + Rectangle	Rectangle hiển thị thật
Thực nghiệm 3	Window + Rectangle + Color	Màu hiển thị đúng
Thực nghiệm 4	Window + Button	Button hiển thị thật
Thực nghiệm 5	Window + Button + Text	Text hiển thị thật

Mỗi thực nghiệm phải được chạy trước khi tiếp tục sang thực nghiệm tiếp theo.

25. Các file thực nghiệm

Một số file test/experiment ban đầu có thể được sử dụng:

```
examples/
├── basic.fxl
├── autoset.fxl
├── nested.fxl
├── preview_test.fxl
├── window_test.fxl
├── rectangle_test.fxl
├── button_test.fxl
└── color_test.fxl
```

Các file này dùng để kiểm tra runtime và rendering chứ không thay thế language specification.

26. Ví dụ thực nghiệm cơ bản

Chương trình:

```FXL
import ui

#autoSet {
    window = Window(
        Size(800, 600)

        rectangle = Rectangle(
            Position(20, 20)
            Size(400, 250)
            Color(#202020)
            Opacity(100%)
        )

        button = Button(
            Position(100, 300)
            Size(200, 60)
            Text("Hello from FXL")
        )
    )
}
```

Pipeline:

```
FXL Source
    ↓
Lexer
    ↓
Parser
    ↓
AST
    ↓
Semantic Analysis
    ↓
Auto Resolution
    ↓
C++ Code Generator
    ↓
Generated C++
    ↓
C++ Compiler
    ↓
FXL Runtime
    ↓
Window
    ↓
Rectangle / Button
    ↓
Renderer
    ↓
Displayed UI
```

Đây là một chương trình thực nghiệm cơ bản của Mục 2.

27. Object Tree và Rendering

Renderer phải dựa trên cấu trúc object tree. Ví dụ:

```
Window
└── Panel
    ├── Rectangle
    └── Button
```

Renderer phải có khả năng duyệt hoặc xử lý tree để xác định các object cần được hiển thị. Cấu trúc object và thứ tự hiển thị sẽ tiếp tục được đặc tả chi tiết khi FXL xây dựng hệ thống layout và rendering nâng cao.

28. Runtime không thay thế Language Specification

Mục 2 mô tả behavior runtime cần thiết để FXL có thể hiển thị UI. Runtime không được tự ý tạo ra cú pháp FXL mới. Ví dụ, nếu language specification chưa định nghĩa:

```
Anchor(...)
Grid(...)
Margin(...)
Padding(...)
```

thì Runtime không được tự coi các tên trên là syntax chính thức của FXL. Các tính năng đó chỉ được bổ sung khi specification tương ứng được thiết kế.

29. Hiệu năng

FXL hướng tới việc sử dụng các thành phần native và graphics system phù hợp với nền tảng. Runtime nên ưu tiên:

- Native execution.
- Các lời gọi native phù hợp.
- Rendering tăng tốc phần cứng khi backend hỗ trợ.
- Hạn chế abstraction không cần thiết trong những phần có ảnh hưởng trực tiếp đến hiệu năng.

Tuy nhiên Mục 2 ưu tiên mục tiêu:

> ***Đúng → Chạy thật → Quan sát được → Sau đó tối ưu***

Không tối ưu hóa trước khi behavior cơ bản hoạt động ổn định.

30. Ranh giới của Mục 2

Mục 2 tập trung vào:

- UI Runtime cơ bản.
- Window.
- Object Tree.
- Rectangle.
- Button.
- Position.
- Size.
- Color.
- Opacity.
- Text.
- Rendering cơ bản.
- Graphics Backend abstraction.
- Preview UI thật.

Các hệ thống sau chưa thuộc phạm vi hoàn chỉnh của Mục 2:

- Advanced Layout.
- Animation.
- Full Event System.
- Accessibility.
- Theme System.
- 3D Rendering hoàn chỉnh.
- Advanced Shader System.
- Hot Reload.
- Visual UI Designer.

- IDE hoàn chỉnh.

Các hệ thống này sẽ được thiết kế ở những mục specification tiếp theo.

31. Trạng thái của Mục 2

Mục 2 xác định nền tảng cho việc đưa FXL từ compiler/runtime sơ khai sang một hệ thống UI có khả năng hiển thị giao diện thật. Đã xác định về mặt specification:

- FXL UI Runtime.
- UI Object.
- Object Tree.
- Window.
- Panel.
- Rectangle.
- Button.
- Position.
- Size.
- Color.
- Opacity.
- Text.
- Rendering System.
- Graphics Backend.
- Preview.
- Chạy thực nghiệm từng bước.
- Quan hệ giữa Compiler, Runtime và Rendering.

Mục tiêu thực nghiệm của Mục 2 là:

```
FXL Source
    ↓
Compiler
    ↓
Generated C++
    ↓
FXL UI Runtime
    ↓
Renderer
    ↓
Native Window
    ↓
Visible UI
```

Mục 2 được xem là hoàn thành khi FXL có thể sử dụng compiler hiện tại để chạy một chương trình FXL và **thực sự hiển thị được giao diện UI cơ bản trên màn hình**, thay vì chỉ ghi lại các thao tác UI dưới dạng console output.

32. Ví dụ kết quả cuối của Mục 2

Một chương trình FXL:

```fxl
import ui

#autoSet {
    window = Window(
        Size(800, 600)

        rectangle = Rectangle(
            Position(20, 20)
            Size(400, 250)
            Color(#202020)
        )

        button = Button(
            Position(100, 300)
            Size(200, 60)
            Text("Hello from FXL")
        )
    )
}
```

sau khi chạy:

```bash
fxlc preview examples/preview_test.fxl
```

phải tạo ra:

```
FXL Source
    ↓
FXL Compiler
    ↓
C++
    ↓
FXL Runtime
    ↓
Rendering
    ↓
Native Window
    ↓
Rectangle + Button + Text
    ↓
Hình ảnh UI thật trên màn hình.
```

Đây là mục tiêu trung tâm của **Mục 2 — FXL UI Runtime & Rendering**.

FXL

Đặc tả ngôn ngữ lập trình

Mục 3: FXL CLI

Thuộc tính	Giá trị
Tên mục	FXL CLI
Mục tiêu	Định nghĩa giao diện dòng lệnh chính thức của FXL Compiler
Executable	`fxlc`
Vai trò	Compiler CLI và công cụ phát triển cơ bản của FXL

1. Giới thiệu về FXL CLI

FXL CLI là giao diện dòng lệnh chính thức của FXL Compiler. Executable chính:

```
fxlc
```

FXL CLI cho phép developer:

- Biên dịch FXL thành C++.
- Build FXL thành executable.
- Preview chương trình FXL.
- Kiểm tra Tokens và AST.
- Chọn Debug hoặc Release.
- Giữ generated C++ để kiểm tra.
- Nhận diagnostic và exit code từ quá trình thực thi.

Pipeline tổng quát:

```
FXL Developer
    ↓
fxlc
    ↓
FXL Compiler
    ↓
C++ Generator
    ↓
C++ Compiler
    ↓
Executable
    ↓
FXL Runtime
    ↓
Application / UI
```

2. Mức độ bắt buộc của specification

Để tránh nhầm lẫn giữa các tính năng bắt buộc và các tính năng mở rộng, FXL CLI sử dụng các mức sau.

2.1. MUST

`MUST` nghĩa là thành phần **bắt buộc**. Một implementation được coi là tương thích với CLI specification của FXL chỉ khi thành phần đó được hỗ trợ đúng behavior được định nghĩa. Ví dụ `fxlc compile`, `fxlc build` và `fxlc preview` là MUST. Nếu implementation không có một command MUST, implementation chưa đáp ứng CLI specification tương ứng.

2.2. SHOULD

`SHOULD` nghĩa là **nên hỗ trợ**, nhưng implementation có thể chưa hỗ trợ trong một phiên bản cụ thể nếu có lý do kỹ thuật rõ ràng. Một tính năng SHOULD không được coi là điều kiện tối thiểu để FXL CLI core hoạt động. Ví dụ có thể bao gồm `--help` và `--version` nếu chúng chưa được hoàn thiện trong phiên bản đầu.

2.3. MAY

`MAY` nghĩa là **tùy chọn**. Implementation có thể hỗ trợ hoặc không hỗ trợ mà vẫn được coi là tương thích với phần CLI core. Ví dụ: additional diagnostic formatting, additional toolchain selection options. Những thứ này không thuộc CLI core nếu specification chưa chuyển chúng thành MUST.

2.4. NOT DEFINED

`NOT DEFINED` nghĩa là behavior **chưa được specification định nghĩa**. Implementation:

- Không được coi behavior đó là FXL CLI standard.
- Không được dùng behavior đó làm yêu cầu bắt buộc đối với developer.
- Có thể triển khai như extension riêng, nhưng extension phải được phân biệt với CLI core.

Ví dụ, `fxlc hot-reload` chưa được định nghĩa thì không phải command chính thức của FXL.

3. Bảng mức độ bắt buộc của CLI

Thành phần	Mức
`fxlc`	**MUST**
`compile`	**MUST**
`build`	**MUST**
`preview`	**MUST**
Input `.fxl`	**MUST**
`-o`	**MUST**
`--debug`	**MUST**
`--release`	**MUST**
`--keep-cpp`	**MUST**
`--tokens`	**MUST**
`--ast`	**MUST**
Diagnostic	**MUST**
Exit code	**MUST**
`--help`	**SHOULD**
`--version`	**SHOULD**
Toolchain selection mở rộng	**MAY**
Command chưa được specification	**NOT DEFINED**

4. Tên executable

FXL CLI MUST sử dụng executable `fxlc`. Trên Windows executable có thể có tên file `fxlc.exe`. Trong specification luôn sử dụng tên `fxlc`.

5. Cấu trúc lệnh

Cấu trúc cơ bản:

```
fxlc <command> <input> [options]
```

Implementation MUST hỗ trợ các command core được định nghĩa trong specification.

6. Command Core

FXL CLI core gồm ba command bắt buộc:

```
fxlc compile
fxlc build
fxlc preview
```

Ba command này MUST tồn tại. Không được thay thế chúng bằng các tên khác trong CLI core. Ví dụ:

```bash
fxlc compile main.fxl
fxlc build main.fxl
fxlc preview main.fxl
```

7. compile

7.1. Mức bắt buộc

`compile` là **MUST**.

7.2. Cú pháp

```bash
fxlc compile <input.fxl>
```

7.3. Mục đích

`compile` MUST thực hiện FXL Compiler Pipeline và tạo generated C++ source.

```
FXL
    ↓
Lexer
    ↓
Parser
    ↓
AST
    ↓
Semantic Analysis
    ↓
Auto Resolution
    ↓
C++ Code Generator
    ↓
C++
```

`compile` không bắt buộc phải tạo executable.

8. build

8.1. Mức bắt buộc

`build` là **MUST**.

8.2. Cú pháp

```bash
fxlc build <input.fxl>
```

8.3. Mục đích

`build` MUST:

- Compile FXL.
- Generate C++.

- Invoke C++ compiler.
- Produce executable nếu toàn bộ quá trình thành công.

Pipeline:

```
FXL
    ↓
FXL Compiler
    ↓
C++
    ↓
C++ Compiler
    ↓
Executable
```

9. preview

9.1. Mức bắt buộc

`preview` là **MUST**.

9.2. Cú pháp

```bash
fxlc preview <input.fxl>
```

9.3. Mục đích

`preview` MUST compile, build và launch chương trình FXL để developer quan sát kết quả. Đối với ứng dụng UI:

```
FXL
    ↓
Compiler
    ↓
C++
    ↓
C++ Compiler
    ↓
Executable
    ↓
FXL Runtime
    ↓
Renderer
    ↓
UI
```

Preview MUST sử dụng Runtime thật. Preview không được chỉ in log để giả lập UI.

10. Input File

Các command `compile`, `build` và `preview` MUST nhận một FXL source file. Input file thông thường có extension `.fxl`. Ví dụ:

```bash
fxlc compile main.fxl
```

Nếu input không tồn tại, command MUST thất bại.

11. -o

11.1. Mức bắt buộc

`-o` là **MUST** cho các command có output file trực tiếp. Implementation MUST hỗ trợ:

```bash
fxlc compile main.fxl -o generated.cpp
```

và:

```bash
fxlc build main.fxl -o MyApp.exe
```

11.2. compile

Trong `compile`, `-o <file>` MUST chỉ định generated C++ output. Ví dụ:

```bash
fxlc compile main.fxl -o generated.cpp
```

11.3. build

Trong `build`, `-o <file>` MUST chỉ định executable output. Ví dụ:

```bash
fxlc build main.fxl -o MyApp.exe
```

11.4. preview

Preview không bắt buộc hỗ trợ output executable bằng `-o`. Preview được phép sử dụng temporary executable. Do đó `-o` trong preview là **MAY**, trừ khi một specification sau này quy định khác.

12. --debug

12.1. Mức bắt buộc

`--debug` là **MUST** đối với `build`. Ví dụ:

```bash
fxlc build main.fxl --debug
```

`--debug` cũng là **MUST** đối với `preview` nếu preview hỗ trợ lựa chọn build configuration. Ví dụ:

```bash
fxlc preview main.fxl --debug
```

12.2. Ý nghĩa

Debug configuration MUST dành cho development và debugging. Có thể bao gồm:

- Debug information.
- Optimization thấp hơn Release.
- Runtime diagnostics.
- Compiler/debugging information cần thiết.

Các chi tiết phụ thuộc C++ toolchain có thể khác nhau.

13. --release

13.1. Mức bắt buộc

`--release` là **MUST** đối với `build`. Ví dụ:

```bash
fxlc build main.fxl --release
```

`--release` cũng là **MUST** đối với `preview` nếu preview hỗ trợ lựa chọn configuration:

```bash
fxlc preview main.fxl --release
```

13.2. Ý nghĩa

Release configuration dành cho executable hướng tới sử dụng/phân phối. Có thể bao gồm:

- Optimization.
- Loại bỏ debug information không cần thiết.
- Build configuration phù hợp cho distribution.

Debug và Release MUST không thay đổi semantic behavior của FXL.

14. Trạng thái mặc định của Build

Nếu `build` không có `--debug` hoặc `--release`, implementation MUST sử dụng một default configuration được specification xác định. Trong phiên bản CLI hiện tại, default nên là **Debug**. Do đó:

```bash
fxlc build main.fxl
```

được hiểu tương đương:

```bash
fxlc build main.fxl --debug
```

Default này phải được documentation và implementation giữ thống nhất.

15. Không được dùng đồng thời Debug và Release

Các option `--debug` và `--release` là hai configuration loại trừ nhau. Lệnh:

```bash
fxlc build main.fxl --debug --release
```

MUST bị từ chối. Ví dụ diagnostic:

```
error: --debug and --release cannot be used together
```

Không được tự chọn một option và bỏ qua option còn lại.

16. --keep-cpp

16.1. Mức bắt buộc

`--keep-cpp` là **MUST**.

16.2. Mục đích

Option này yêu cầu giữ generated C++ sau quá trình build. Ví dụ:

```bash
fxlc build main.fxl --debug --keep-cpp
```

Kết quả khái niệm:

```
main.fxl
    ↓
generated.cpp
    ↓
main.exe
```

Generated C++ phải là source thật do C++ Code Generator tạo.

17. --tokens

17.1. Mức bắt buộc

`--tokens` là **MUST**.

17.2. Mục đích

Option này cho phép developer kiểm tra output của Lexer. Ví dụ:

```bash
fxlc compile main.fxl --tokens
```

Output MUST dựa trên tokens thực tế được Lexer tạo. Không được tạo output token giả.

18. --ast

18.1. Mức bắt buộc

`--ast` là **MUST**.

18.2. Mục đích

Option này cho phép developer kiểm tra AST. Ví dụ:

```bash
fxlc compile main.fxl --ast
```

Output MUST phản ánh AST thực tế của Parser. Không được tạo một cây AST chỉ để hiển thị.

19. Kết hợp --tokens và --ast

Implementation MUST cho phép:

```bash
fxlc compile main.fxl --tokens --ast
```

nếu cả hai option được truyền. Output phải phân biệt rõ **Tokens** và **AST**.

20. --help

20.1. Mức bắt buộc

`--help` là **SHOULD** ở CLI version đầu. Nếu được triển khai, lệnh sau MUST hiển thị thông tin usage và command:

```bash
fxlc --help
```

Help của command cũng SHOULD được hỗ trợ:

```bash
fxlc compile --help
fxlc build --help
fxlc preview --help
```

Trong phiên bản CLI tương lai, `--help` có thể được nâng lên MUST.

21. --version

21.1. Mức bắt buộc

`--version` là **SHOULD** ở CLI version đầu. Nếu được triển khai:

```bash
fxlc --version
```

MUST trả về version của FXL Compiler CLI. Version format được định nghĩa bởi versioning policy của FXL.

22. Diagnostic

Diagnostic là **MUST**. CLI MUST báo lỗi khi command không thể hoàn thành. Diagnostic nên có:

```
severity
file
line
column
message
```

nếu thông tin tương ứng tồn tại. Ví dụ:

```
error: examples/main.fxl:12:5
Unknown operation 'Rotate'
```

23. Phân loại lỗi

FXL CLI phải phân biệt các nhóm lỗi chính.

23.1. CLI Error

```
unknown command
missing input
invalid option
missing option value
```

23.2. FXL Compile Error

```
Lexer error
Parser error
Semantic error
Auto Resolution error
```

23.3. C++ Build Error

Xảy ra khi C++ compiler không build được generated C++.

23.4. Runtime Error

Xảy ra sau khi executable bắt đầu chạy.

Các nhóm lỗi này không được gộp thành một loại duy nhất khi implementation có đủ thông tin để phân biệt.

24. Exit Code

Exit code là **MUST**. Quy tắc tối thiểu:

```
0     = success
!= 0  = failure
```

- Nếu `fxlc compile main.fxl` thành công → exit code `0`.
- Nếu compilation thất bại → exit code khác `0`.
- Nếu `fxlc preview main.fxl` không build hoặc không thể launch theo yêu cầu → exit code khác `0`.

Các mã lỗi chi tiết hơn MAY được chuẩn hóa sau.

25. stdout và stderr

CLI SHOULD phân biệt output thông thường và error output. Thông tin như sau là normal output:

```
Compiled to C++.
Built executable.
Launching preview...
```

Error và diagnostic SHOULD được xuất qua error stream. Điều này giúp FXL CLI hoạt động tốt với:

- Terminal.
- IDE.
- Script.
- Build system.
- CI/CD.

26. Working Directory

Path tương đối MUST được tính từ working directory của process. Ví dụ, khi working directory là:

```
C:/Projects/FXL
```

và lệnh:

```bash
fxlc compile examples/basic.fxl
```

tham chiếu:

```
C:/Projects/FXL/examples/basic.fxl
```

CLI không được giả định working directory luôn là thư mục chứa `fxlc`.

27. C++ Compiler Backend

FXL Compiler target ban đầu là C++. FXL CLI MUST sử dụng một C++ compiler backend để build executable. Ví dụ trên Windows có thể là `g++` hoặc `cl.exe`.

FXL CLI không được giả định rằng mọi hệ thống chỉ có một C++ toolchain. Nếu implementation hỗ trợ nhiều backend, cách lựa chọn backend MAY được mở rộng bằng configuration hoặc CLI option trong specification tương lai.

28. Không được thay đổi FXL Semantic theo Toolchain

Việc thay đổi C++ compiler backend (`g++` hoặc `cl.exe`) không được làm thay đổi semantic behavior của FXL. Pipeline FXL vẫn là:

```
Lexer
    ↓
Parser
    ↓
AST
    ↓
Semantic
    ↓
Auto Resolution
    ↓
C++ Codegen
```

29. Command Extension

Một implementation MAY cung cấp command riêng ngoài CLI core. Ví dụ:

```
fxlc clean
fxlc package
fxlc install
```

Tuy nhiên các command extension:

- Không phải FXL CLI core.
- Không được coi là MUST của FXL.
- Không được làm thay đổi behavior của command core.
- Phải được documentation riêng nếu được cung cấp.

30. Reserved Command

Nếu một command chưa được specification định nghĩa (**NOT DEFINED**) thì compiler không được tự coi command đó là command chính thức của FXL. Ví dụ:

```bash
fxlc hot-reload
```

chưa phải command FXL core nếu chưa được một specification khác định nghĩa.

31. Cú pháp CLI chuẩn

CLI grammar:

```
command
    ::= compile-command
      | build-command
      | preview-command
      | help-command
      | version-command

compile-command
    ::= "compile" input-file compile-option*

build-command
    ::= "build" input-file build-option*

preview-command
    ::= "preview" input-file preview-option*

input-file
    ::= path-to-fxl-file

compile-option
    ::= "-o" output-cpp
      | "--tokens"
      | "--ast"

build-option
    ::= "-o" output-executable
      | "--debug"
      | "--release"
      | "--keep-cpp"

preview-option
    ::= "--debug"
      | "--release"
```

Lưu ý: `help` và `version` được mô tả ở đây vì chúng là CLI convention, nhưng trong CLI version đầu chúng có mức `SHOULD` theo mục 20 và 21.

32. Ví dụ sử dụng CLI

Mô tả	Lệnh	Mức
Compile	`fxlc compile examples/basic.fxl`	**MUST**
Compile với output	`fxlc compile examples/basic.fxl -o build/basic.cpp`	**MUST**
Xem Tokens	`fxlc compile examples/basic.fxl --tokens`	**MUST**
Xem AST	`fxlc compile examples/basic.fxl --ast`	**MUST**
Debug Build	`fxlc build examples/basic.fxl --debug`	**MUST**
Release Build	`fxlc build examples/basic.fxl --release`	**MUST**
Giữ generated C++	`fxlc build examples/basic.fxl --debug --keep-cpp`	**MUST**
Preview	`fxlc preview examples/basic.fxl`	**MUST**
Preview Debug	`fxlc preview examples/basic.fxl --debug`	**MUST**
Preview Release	`fxlc preview examples/basic.fxl --release`	**MUST**
Help	`fxlc --help`	**SHOULD**
Version	`fxlc --version`	**SHOULD**

33. Workflow chuẩn

Một developer có thể sử dụng:

```
Write FXL
    ↓
fxlc compile
    ↓
Inspect / Fix
    ↓
fxlc build --debug
    ↓
Run
    ↓
fxlc preview
    ↓
Test UI
    ↓
fxlc build --release
    ↓
Distribution
```

Các bước trên sử dụng CLI core.

34. Conformance

Một implementation được coi là **FXL CLI Core Conforming** khi:

```
[MUST]
✓ fxlc
✓ compile
✓ build
✓ preview
✓ -o
✓ --debug
✓ --release
✓ --keep-cpp
✓ --tokens
✓ --ast
✓ Diagnostics
✓ Exit Codes
```

được hỗ trợ đúng specification. Các thành phần `SHOULD` không làm mất tính tương thích CLI core nếu chưa được hỗ trợ. Các extension `MAY` không làm thay đổi CLI core.

35. Nguyên tắc cuối cùng

FXL CLI phải tuân thủ:

Mức	Ý nghĩa
MUST	Thực hiện đúng mọi behavior bắt buộc.
SHOULD	Ưu tiên hỗ trợ nhưng có thể triển khai sau.
MAY	Tùy implementation.
NOT DEFINED	Không được tự coi là behavior chính thức của FXL.

Nguyên tắc này giúp specification không bị nhập nhằng giữa "đây là bắt buộc" và "đây chỉ là một khả năng". Mục 3 vì vậy trở thành chuẩn giao tiếp chính thức giữa developer, FXL Compiler và hệ thống build/runtime.

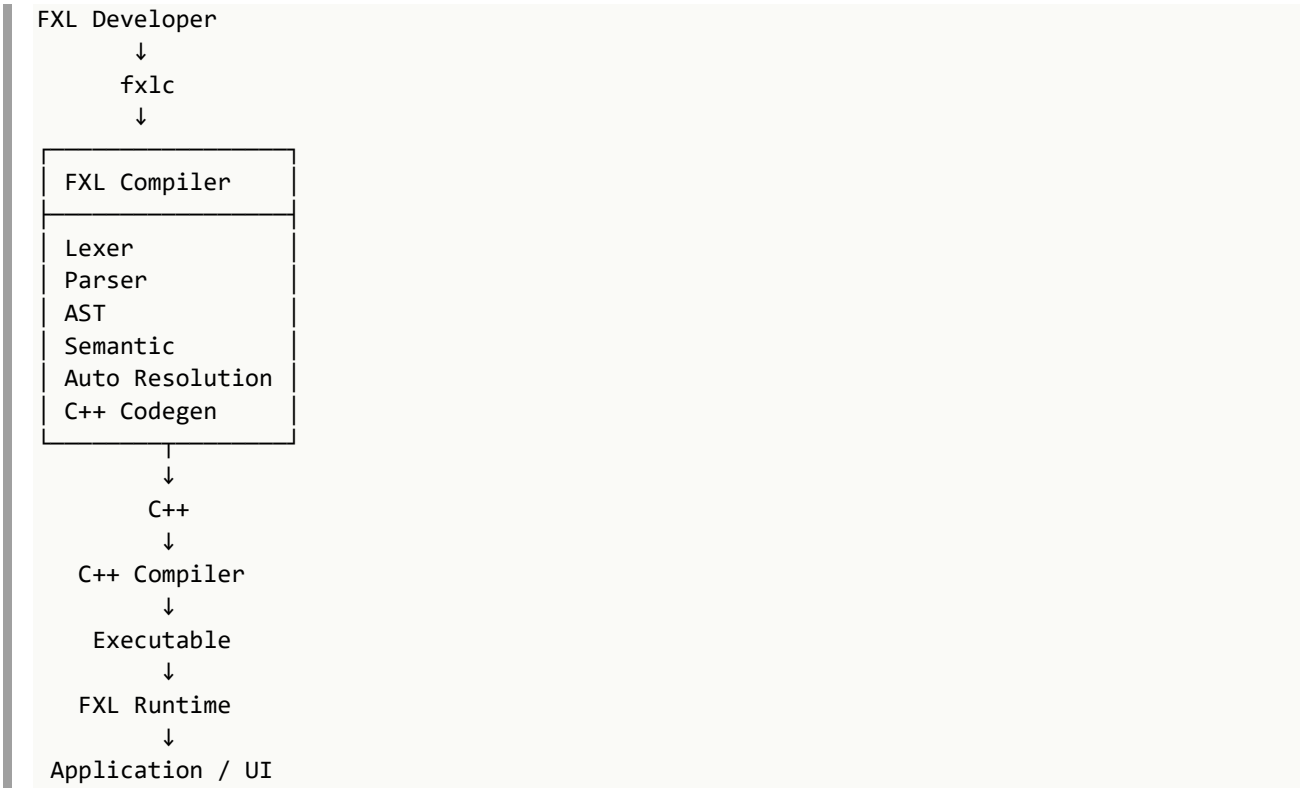
36. Trạng thái của Mục 3

Mục 3 xác định CLI core của FXL:

```
fxlc
├── compile
├── build
└── preview
```

và mức độ bắt buộc của từng thành phần. CLI core là nền tảng để FXL được sử dụng độc lập với IDE.

Pipeline hoàn chỉnh:



Mục 3 là specification về **cách developer điều khiển pipeline này thông qua command line**.